

Seatwork 5.1

Queue - Linked List Application

Course Code: CPE010

Program: Computer Engineering

Course Title: Data Structures and Algorithms

Date Performed: Sep 9, 2025

Section: CPE21S4

Date Submitted: Sep 9, 2025

Name(s): Punay, Heidee S.

Instructor: Engr. Jimlord Quejado

6. Output

```
1  #include<iostream>
2  #include "queue.h"
3
4  int main()
5  {
6
7      Queue<std::string>Q1;
8
9
10     Q1.enqueue("francis");
11     Q1.enqueue("Jason");
12     Q1.enqueue("Curwin");
13     Q1.enqueue("Abila");
14     Q1.enqueue("Dano");
15     Q1.getFront();
16     Q1.dequeue();
17     Q1.getFront();
18     Q1.getRear();
19     Q1.display();
20
21
22
23
24
25
26     return 0;
27 }
```

```
C:\Users\TIPQC\Documents\q X + v
A Queue Has Been Created.
Enqueue to an Empty Queue
Successfully Enqueued
Successfully Enqueued
Successfully Enqueued
Successfully Enqueued
Current Front: francis
Current Front: Jason
Current Rear: Dano
Jason
Curwin
Abila
Dano

-----
Process exited after 0.01348 seconds with return value 0
Press any key to continue . . . |
```

7. Supplementary Activity

```
#ifndef QUEUE_H
#define QUEUE_H
#include <iostream>

template<typename T>
class Node{
public:
    T data;
    Node *next;
    Node(T new_data){
        data = new_data;
        next = nullptr;
    }
};
```

Analysis:

Since it is a linked list we need to define the data and to the new data where the next in the queue will be equal to null pointer. The template<typename T> indicates to us that we didn't know what data type is the data. We created a class Node to make the queue function as a linked list.

```
template <typename T>
class Queue{
private:
    Node<T> *front;
    Node<T> *rear;
```

Analysis:

We did another template<typename T> because the first one was for the class of Node. In this function, this time it is a class of queue to define the front and rear of the queue as a linked list.

```
public:
//create an empty queue
Queue(){
    front = rear = nullptr;
    std::cout<<"A Queue Has Been Created."<<std::endl;
}
```

Analysis:

This function creates the queue where front is equal to rear and to null pointer. This creates an empty queue.

```
//isEmpty
bool isEmpty()
{
    return front == nullptr;
}
```

Analysis:

This function checks if the queue is empty, we can also identify it with the "return" that front is equal to null pointer. It indicates to us that the queue is empty.

```
//enqueue
void enqueue(T new_data){
    Node<T> *new_node = new Node<T>(new_data);

    if(isEmpty()){
        front = rear = new_node;
        std::cout<<"Enqueue to an Empty Queue"<<std::endl;
        return;
    }

    rear->next = new_node;
    rear = new_node;
    std::cout<<"Successfully Enqueued"<<std::endl;
}
```

Analysis:

This function adds new data into the queue where the new node is the next data to the queue. It checks first if the queue is empty and if it is, we add a new data to the queue.

```

//dequeue
void dequeue(){
    if(isEmpty()){
        return;
    }
    // storing the front to a temporary pointer
    Node<T> *temp = front;
    //check if after the dequeue, the queue is empty
    if(front == nullptr) rear == nullptr;

    else{
        //reassign the front to the next node
        front = front->next;
    }
    delete temp;
}

```

Analysis:

This function deletes data in the queue. First, it checks if the queue is empty and stores the front data to a temporary pointer. After, it checks the queue is empty after dequeue and deletes the data in the temporary pointer

```

//getfront
void getFront(){
    if(isEmpty()){
        std::cout<<"The Queue is MT"<<std::endl;
        return;
    }

    std::cout<<"Current Front: "<<front->data<<std::endl;
}

```

Analysis:

This function gets the front. First, it checks if the queue is empty again. It prints the current front where front is pointed to the data.

```

//getrear
void getRear(){
    if(isEmpty()){
        std::cout<<"The Queue is MT"<<std::endl;
        return;
    }
    std::cout<<"Current Rear: "<< rear->data<<std::endl;
}

```

Analysis:

This function gets the rear data. Again, it checks the queue if it is empty and prints the current rear where it is pointed to the data.

```

//display
void display(){
    if(isEmpty()){
        std::cout<<"The Queue is MT"<<std::endl;
    }
    Node<T> *temp=front;
    while(temp !=nullptr){
        std::cout<<temp->data<<std::endl;
        temp = temp->next;
    }
    std::cout<<std::endl;
}

```

Analysis:

This function shows the data that were enqueued in the queue.

```

//to deollocate memory
~Queue(){
    while(!isEmpty()){
        dequeue();
    }
}
};

```

Analysis:

This function deallocates the memory of the queue.

8. Conclusion

To conclude, We did the queue and learned the linked lists again. It was like a recap for pointers also and to other functions – and also to basic c++ programming. Overall, we did a queue implemented to the linked list and how queue works.

9. Assessment Rubric